

WEEK 5 : LECTURE 4 - CLASS DIAGRAMS

Bachelor's Degree in Information Technology
BITSD4111, SOFTWARE DESIGN

Trimester: 06

From **March 2024** to **June 2024**

OBJECTIVES

- Associations
- The generalize relationship
- More about attributes and operations.
- THE ATM case study

ASSOCIATIONS

- The steps during object modeling
 - 1. Class identification
 - Based on the fundamental assumption that we can find abstractions
 - 2. Find the attributes
 - 3. Find the operations/methods
 - 4. Find the associations between classes
- The Order of steps secondary, only a heuristic
- Iteration is important

ASSOCIATIONS

- Three broad relationships exist in class diagrams
 - 1. Association
 - 2. Composition / Aggregation
 - 3. Generalization / Inheritance
- Note that composition/aggregation are specific cases of association

ASSOCIATIONS

- In class diagrams, associations represent the connections and interactions between different classes, providing insight into
- If two classes in a model need to communicate with each other, there must be a link between them, and that can be represented by an association (connector) their relationships.
- These associations define the links between classes and demonstrate the various connections, such as one-to-one, one-to-many, or many-to-many.

ASSOCIATIONS

- In UML an association shows a relationship between two or more classes.
- Association can be represented by a line between these classes with an arrow indicating the navigation direction.
 - In case an arrow is on both sides (or no arrow at all), the association is known as a bidirectional association.

ASSOCIATIONS

PROPERTIES OF ASSOCIATIONS

- 1. name -- an association can have a name, which describes the nature of the association
 - The name is optional, but is a good idea when the nature of the association is not clear
 - When the association is one of our special types -- aggregation or inheritance -- the name is usually not needed, as the nature of the association is clear already
 - With other general associations, the name is a good idea to include

ASSOCIATIONS

PROPERTIES OF ASSOCIATIONS

- 2. A role at each end
 - A role name identifies the purpose of the class at a given end of an association
- Again, role names are best to include when the relationship is not clear
- Aggregation and inheritance are, again, usually clear on their own

ASSOCIATIONS

PROPERTIES OF ASSOCIATIONS

- 2. A role at each end
 - When there's only one association between a pair of distinct classes, the class names usually serve as good role names already (so no need to come up with new ones)
 - Good places to use role names:
 - When an association between two objects of the same class type
 - When we have multiple associations between same pair of classes

ASSOCIATIONS

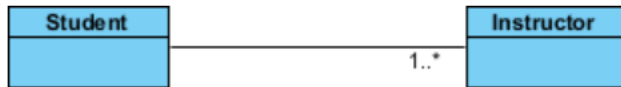
PROPERTIES OF ASSOCIATIONS

- 3. A multiplicity at each end
 - a multiplicity identifies the possible number of instances of one class relating to an instance of the class at the other end
 - Not usually needed with inheritance relationships
 - With aggregation, the assumed multiplicity at the diamond end would be 1 (unless specified otherwise)
 - Qualification can be used to reduce multiplicities (like "0 or more").

ASSOCIATIONS

PROPERTIES OF ASSOCIATIONS

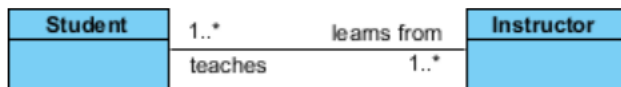
A single student can associate with multiple teachers:



The example indicates that every Instructor has one or more Students:



We can also indicate the behavior of an object in an association (i.e., the role of an object) using role names.



ASSOCIATIONS

PROPERTIES OF ASSOCIATIONS

- With inheritance, the terms generalization and specialization are sometimes used
- Generalization is the activity that starts with specific classes and identifies commonalities that can be factored out (into base categories)
- Specialization is the activity that starts with a higher-level concept and identifies more specific concepts and subcategories from there
- Both activities can help identify appropriate places to set up inheritance relationships

ASSOCIATIONS

IDENTIFYING ASSOCIATIONS

- Check if two classes have any relationship
- Examine verb phrases for this relationship
- Add association and role names where appropriate
- Use qualifiers to reduce multiplicity and identify key attributes
- Eliminate redundant associations (i.e. too many paths between two classes)
- Use generalization and specialization to identify appropriate inheritance relationships

ASSOCIATIONS

IDENTIFYING ATTRIBUTES

- Examine possessive phrases and adjectives
- Entity objects usually have specific information to be stored. Represent the stored state with attributes. You can ask the following:
 - How is the object described?
 - What parts of the description are relevant and applicable to the problem domain?
 - What is the minimal description needed?

ASSOCIATIONS

IDENTIFYING ATTRIBUTES

- If an attribute is also being modeled as an object, don't list as an attribute -- use an aggregation relationship between classes instead
 - Is the independent existence of the attribute important? i.e. might it be manipulated as an entity by itself? Then make it an object
 - Or is only the value of the attribute important in the context of the object it's in? In this case, better to just make an attribute

ASSOCIATIONS

IDENTIFYING ATTRIBUTES

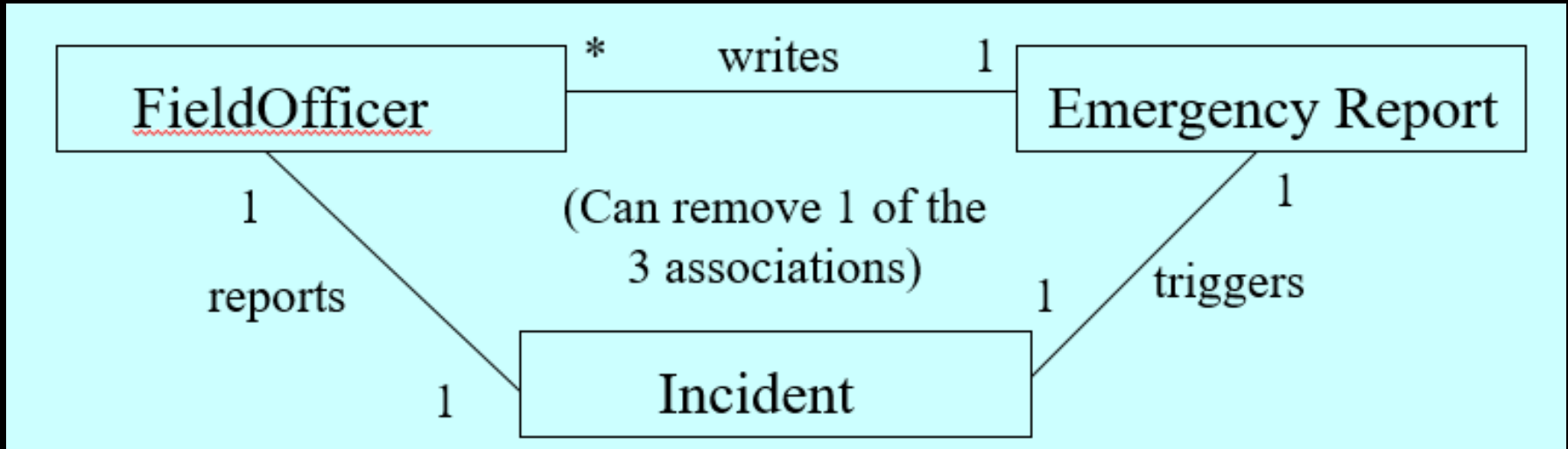
- Example: A Person object, with an address attribute. Will the address be an entity to be manipulated sometimes by itself? If so, make it an object, with an association to the Person. If you only manipulate the address in the context of the Person, make it an attribute.
- Remember, this is analysis, not design. Don't waste time on fine details until the overall structure is stable

ASSOCIATIONS

IDENTIFYING RESPONSIBILITIES

- This already being done when using the CRC card technique
- A responsibility is usually either what a class knows (data) or what a class does (behavior)
- Attributes qualify as responsibilities if the class' job is to store specific data (e.g. a message queue storing saved e-mail messages)
- For behavior, look for verbs in the requirements, use cases, and scenarios

ASSOCIATIONS



ASSOCIATIONS EXAMPLE

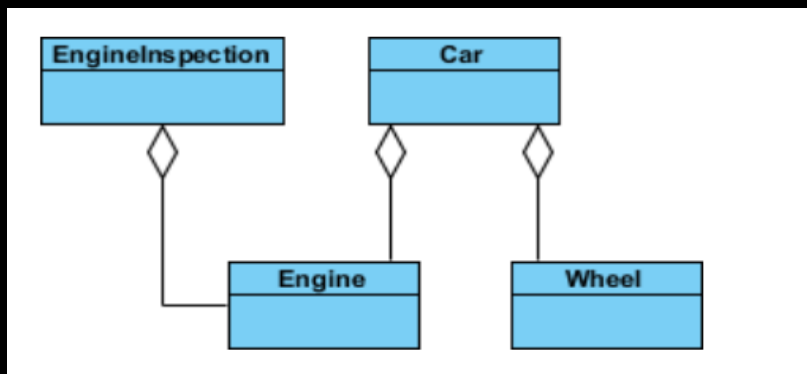
Problem Statement : A stock exchange lists many companies. However, a stock exchange lists only one company with a given ticker symbol. A company may be listed on many stock exchanges, possibly with different ticker symbols. Find company with ticker symbol AAPL.



ASSOCIATIONS : AGGREGATION

- Aggregation and Composition are subsets of association meaning they are specific cases of association.
- In both aggregation and composition object of one class "owns" object of another class. But there is a subtle difference:
 - Aggregation implies a relationship where the child can exist independently of the parent. Example: Class (parent) and Student (child). Delete the Class and the Students still exist.
- Aggregation relationships are shown by a white diamond-shaped arrowhead pointing towards the target or parent

ASSOCIATIONS : AGGREGATION

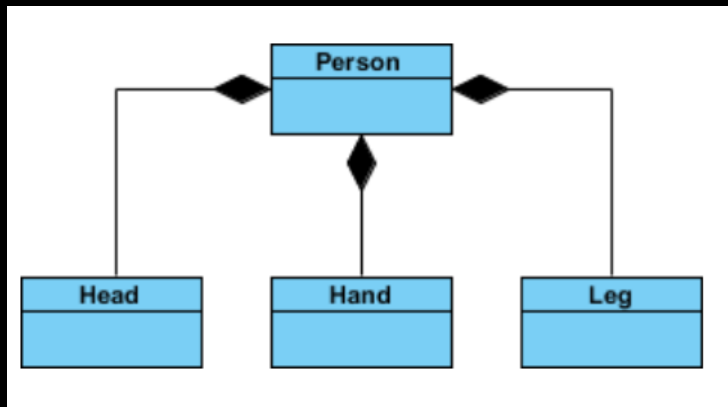


ASSOCIATIONS : COMPOSITION

- Composition implies a relationship where the child cannot exist independent of the parent.
 - Example: House (parent) and Room (child). Rooms don't exist separate to a House.
 - In composition It's important to note that the aggregation link doesn't state in any way that Class A owns Class B nor that there's a parent-child relationship (when parent deleted all its child's are being deleted as a result) between the two. Actually, quite the opposite!

ASSOCIATIONS : COMPOSITION

- The aggregation link is usually used to stress the point that Class A instance is not the exclusive container of Class B instance, as in fact the same Class B instance has another container/s.
- Composition relationships are shown by a black diamond-shaped arrowhead pointing towards the target or parent



GENERALISATION

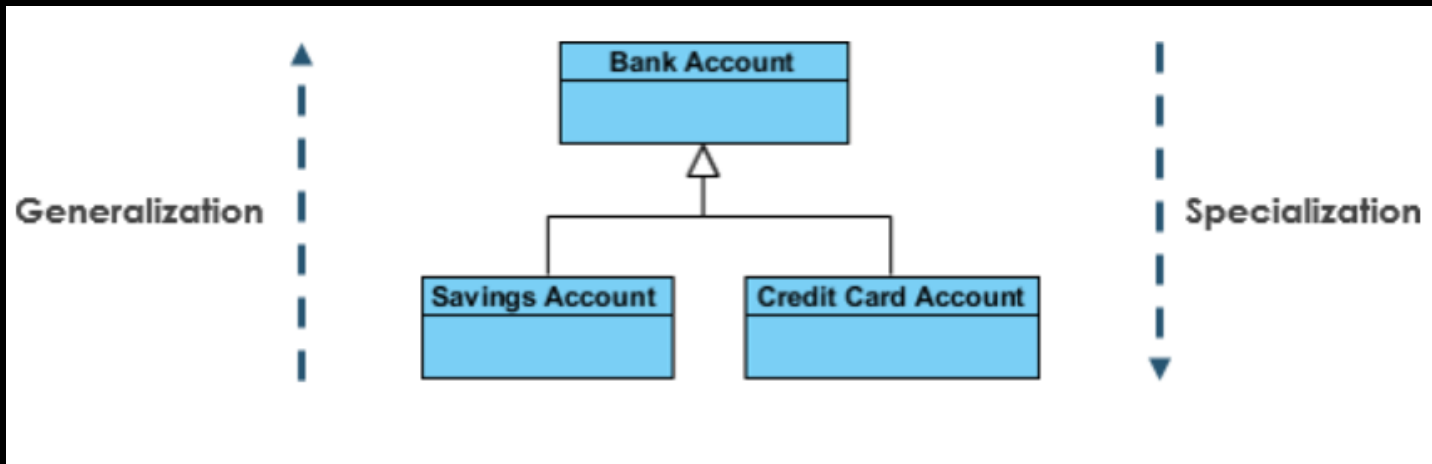
- Generalization (or inheritance) is a mechanism for combining similar classes of objects into a single, more general class.
- Generalization identifies commonalities among a set of entities.
 - The commonality may be of attributes, behavior, or both.
- In other words, a superclass has the most general attributes, operations, and relationships that may be shared with subclasses. A subclass may have more specialized attributes and operations.

GENERALISATION

- Specialization is the reverse process of Generalization means creating new sub-classes from an existing class.
- For Example, a Bank Account is of two types - Savings Account and Credit Card Account. Savings Account and Credit Card Account inherit the common/ generalized properties like Account Number, Account Balance, etc. from a Bank Account and also have their specialized properties like unsettled payment etc.

GENERALISATION

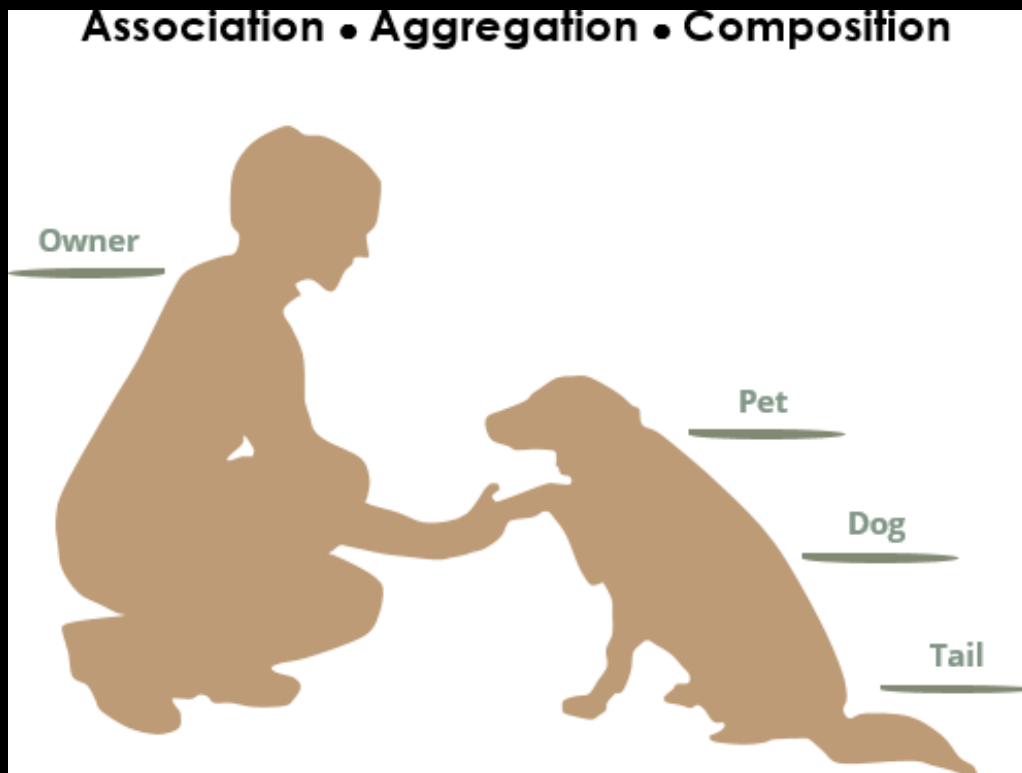
- generalization relationship is also informally called "Is A" relationship
- It is shown as a line with a hollow triangle as an arrowhead between the symbols representing the involved classes



GENERALISATION

- Specialization is the reverse process of Generalization means creating new sub-classes from an existing class.
- For Example, a Bank Account is of two types - Savings Account and Credit Card Account. Savings Account and Credit Card Account inherit the common/ generalized properties like Account Number, Account Balance, etc. from a Bank Account and also have their specialized properties like unsettled payment etc.

ASSOCIATION VS COMPOSITION VS AGGREGATION



ASSOCIATION VS COMPOSITION VS AGGREGATION

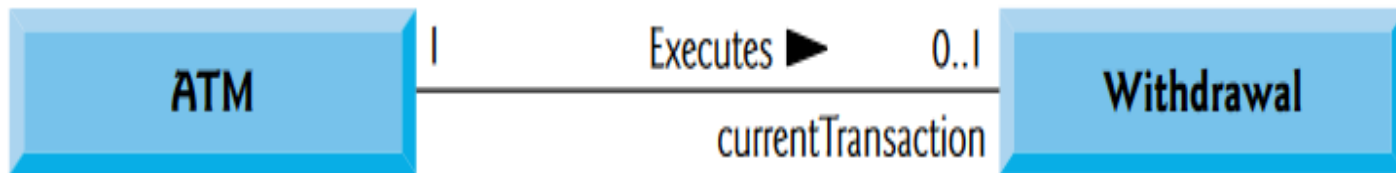
- We see the following relationships:
 - owners feed pets, pets please owners (association)
 - a tail is a part of both dogs and cats (aggregation / composition)
 - a cat is a kind of pet (inheritance / generalization)

THE ATM CASE STUDY

- The figure below shows how our classes ATM & Withdrawal relate to one another.
 - The line that connects the two classes represents an association.
 - Multiplicity values indicate how many objects of each class participate in the association.
 - One ATM object participates in an association with either zero or one Withdrawal objects.
 - currentTransaction is a role name, which identifies the role the Withdrawal object plays.

THE ATM CASE STUDY

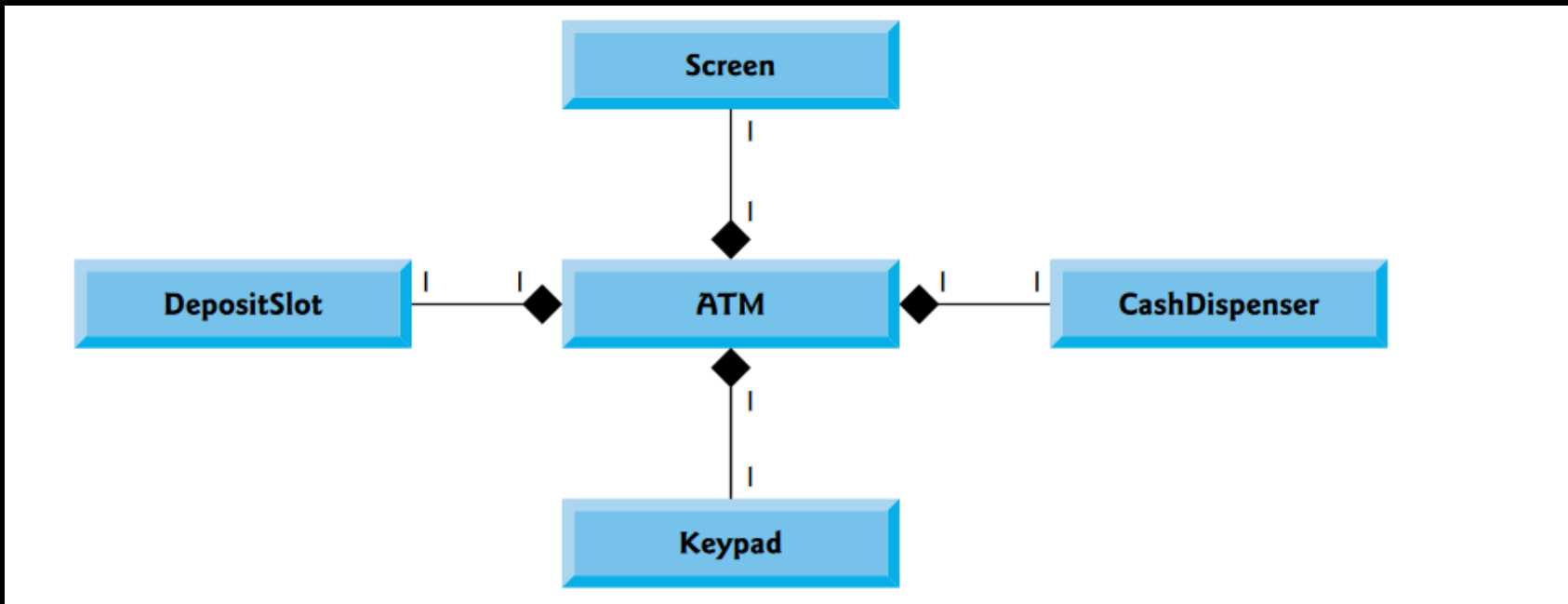
- The figure below shows how our classes ATM & Withdrawal relate to one another.



THE ATM CASE STUDY

- In the figure below, the solid diamonds indicate that class ATM has a composition relationship with classes Screen, Keypad, CashDispenser and DepositSlot.
- Composition implies a whole/part relationship—the ATM “has a” screen, a keypad, a cash dispenser and a deposit slot.
- The has-a relationship defines composition.

THE ATM CASE STUDY



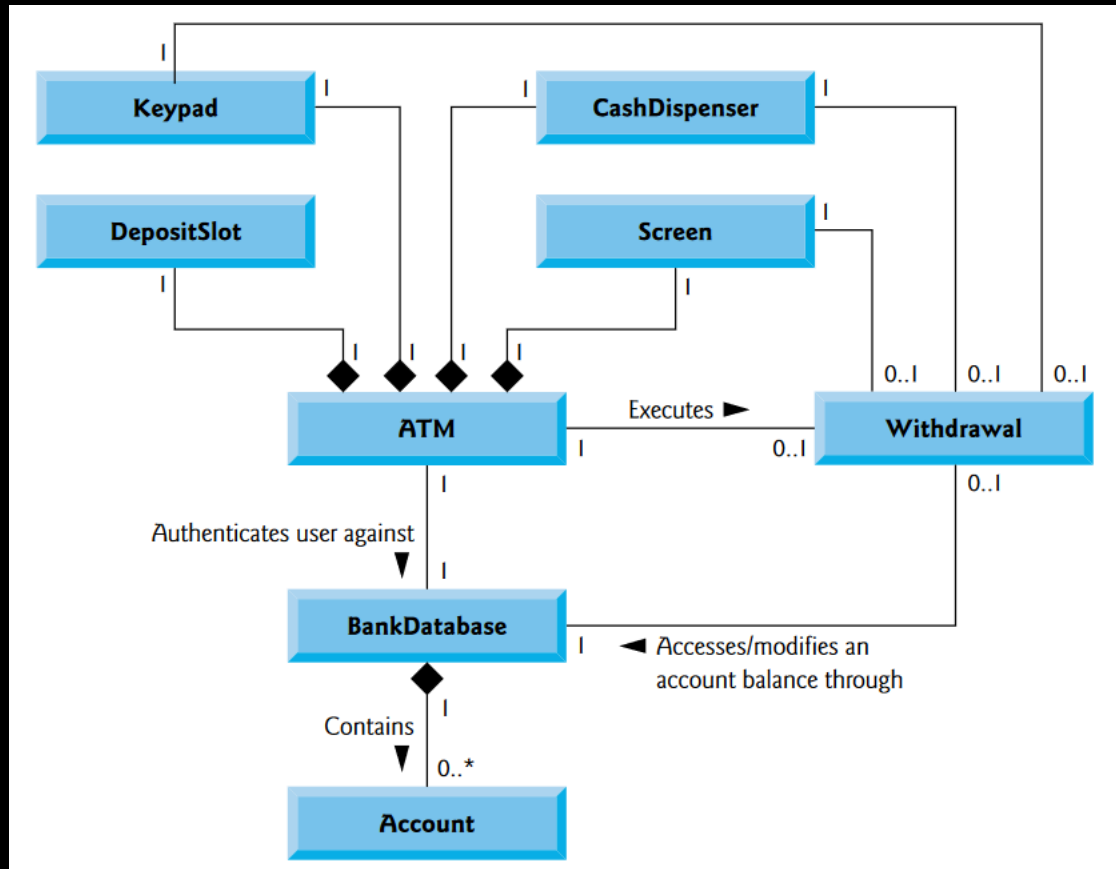
THE ATM CASE STUDY

- Composition relationships have the following properties:
 - – Only one class in the relationship can represent the whole.
 - – The parts in the composition relationship exist only as long as the whole.
 - – A part may belong to only one whole at a time.
- If a “has-a” relationship does not satisfy one or more of these criteria, hollow diamonds are used to indicate

THE ATM CASE STUDY

- The next figure shows a class diagram for the ATM system
- The class diagram shows that class ATM has a one-to-one relationship with class BankDatabase.
- We also model that one object of class BankDatabase participates in a composition relationship with zero or more objects of class Account.

THE ATM CASE STUDY



THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM

- A person's attributes include height, weight and whether the person is left-handed, right-handed or ambidextrous.
- We can identify attributes of the classes in our system by looking for descriptive words and phrases in the requirements document.

THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM

Class	Descriptive words and phrases
ATM	user is authenticated
BalanceInquiry	account number
Withdrawal	account number amount
Deposit	account number amount
BankDatabase	[no descriptive words or phrases]
Account	account number PIN balance
Screen	[no descriptive words or phrases]
Keypad	[no descriptive words or phrases]
CashDispenser	begins each day loaded with 500 \$20 bills
DepositSlot	[no descriptive words or phrases]

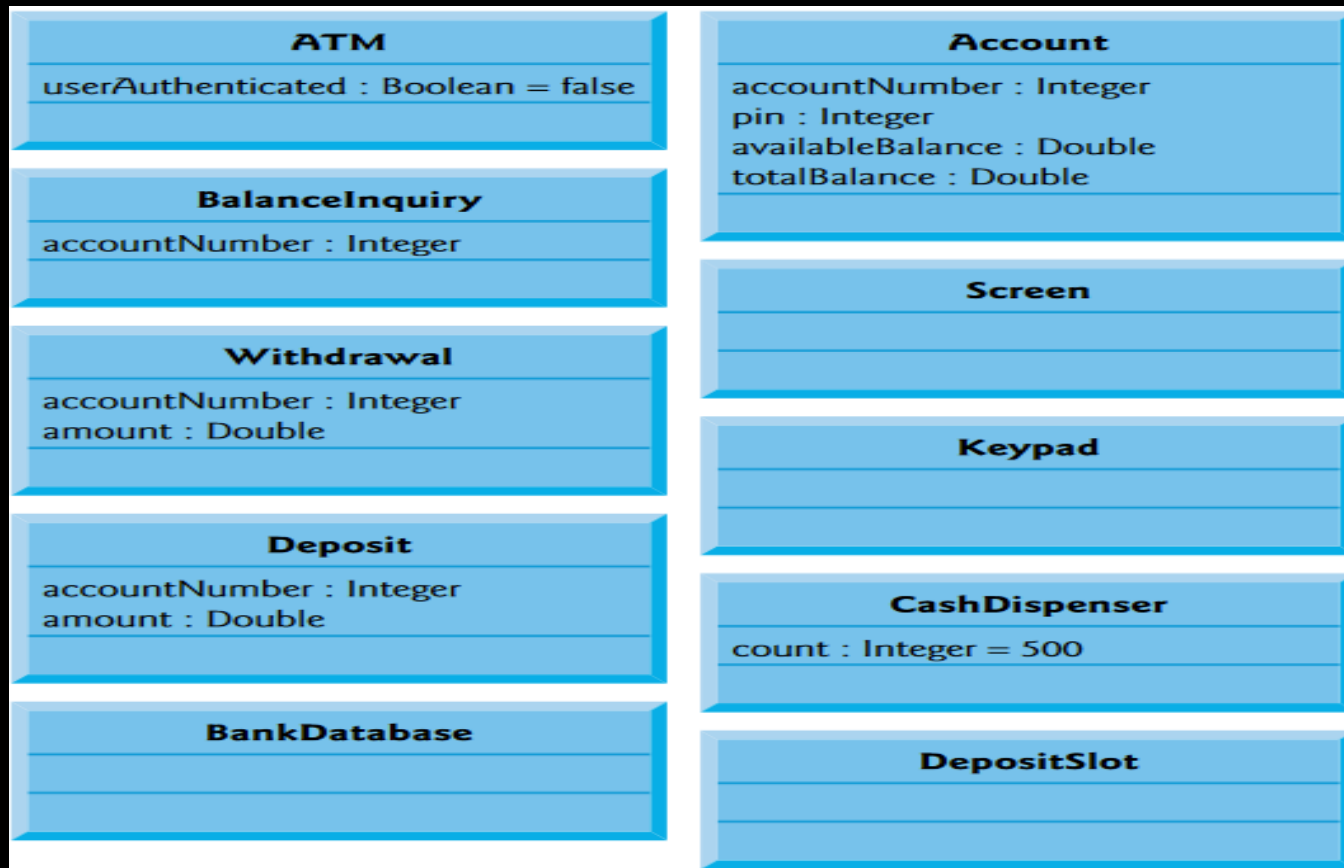
THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM

- The class diagram on the next page lists some of the attributes for the classes in our system.
- We list each attribute's name and type, followed in some cases by an initial value.

THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM



THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM

- Consider the userAuthenticated attribute of class ATM:
userAuthenticated : bool = false
- This attribute declaration contains:
 - – the attribute name, userAuthenticated.
 - – the attribute type, bool.
 - – an initial value for an attribute, false.

THE ATM CASE STUDY

IDENTIFYING CLASS ATTRIBUTES IN THE ATM SYSTEM

- Software Engineering Observation
- Early in the design process, classes often lack attributes (and operations).
 - Such classes should not be eliminated, however, because attributes (and operations) may become evident in the later phases of design and implementation.

WEEK 5 : LECTURE 4 - CLASS DIAGRAMS

Bachelor's Degree in Information Technology
BITSD4111, SOFTWARE DESIGN

Trimester: 06

From **March 2024** to **June 2024**

THE END